WEEK 2 — PROOF DOCUMENT

Serverless Image Processing on AWS

API Gateway + SNS Integration — Screenshot Evidence

Student Name	Sujal Pandey
Project	Serverless Image Processing on AWS
Document Type	Week 2 — Screenshot Proof
Screenshots	7 proofs of implementation
AWS Region	us-east-1 (N. Virginia)
Services Covered	Lambda, API Gateway, S3, SNS, Postman
Total AWS Cost	$0.00 — Free Tier
Date	26-03-2026

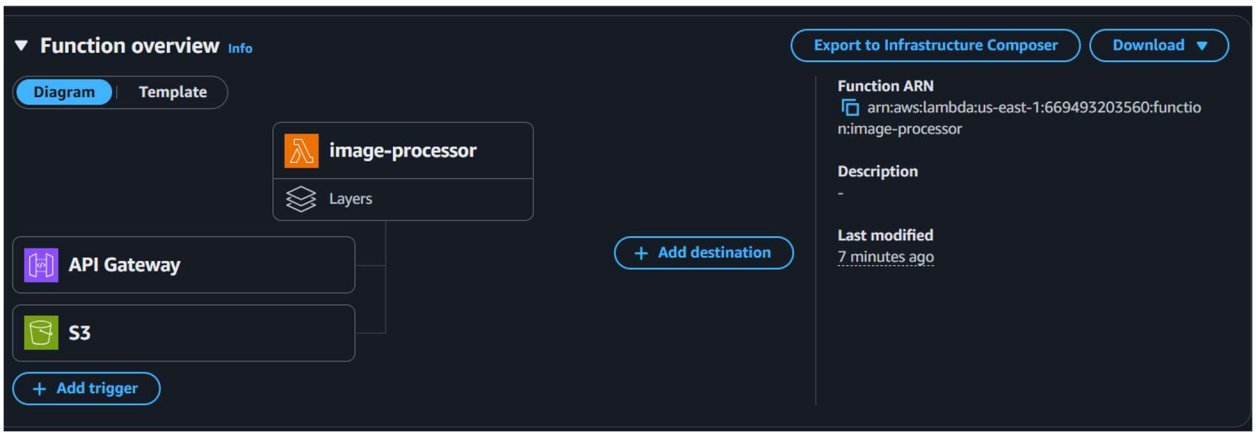
01 Architecture Overview

Where: Lambda Console → image-processor function → scroll to top

The Function Overview diagram is the most important single screenshot of the entire project. It shows at a glance that your Lambda function has two triggers (API Gateway + S3) on the left and one destination (SNS) on the right — proving the complete Week 2 architecture is configured and working.

What this screenshot proves:

- API Gateway is connected as a trigger on the left side of the diagram
- S3 (sujal-input-images) is also connected as a trigger — proving both methods work
- Amazon SNS appears on the right as a destination — proving notifications are set up
- Function name 'image-processor', runtime Python 3.12, region us-east-1 are all visible



Navigation: AWS Console → Lambda → Functions → image-processor → scroll to top of page

What to capture: The entire 'Function overview' box showing triggers on left and destinations on right

02 Environment Variables

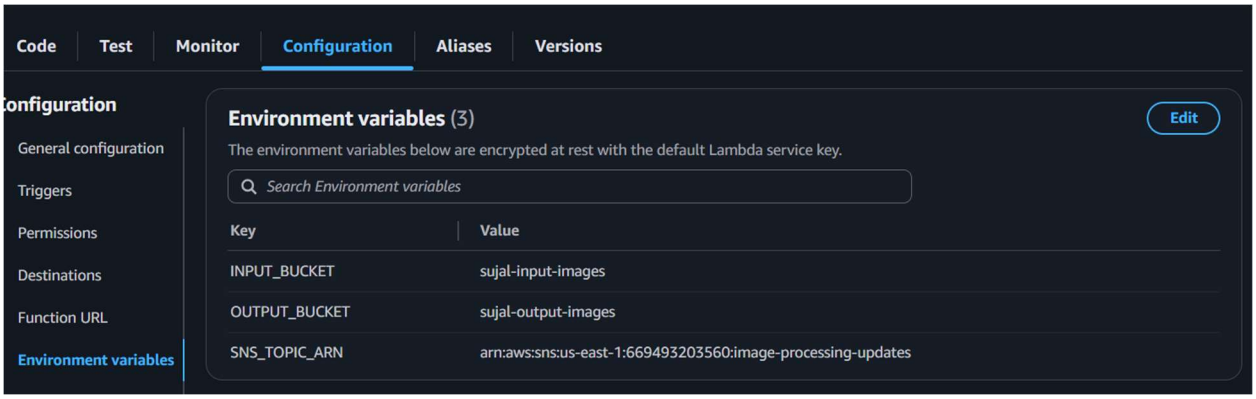
Where: Lambda Console → image-processor → Configuration tab → Environment variables

Environment variables prove that your project is built to professional standards. Instead of hardcoding sensitive values like bucket names and ARN strings directly in the Python code, they are stored externally as configuration. This is how real-world cloud applications are built.

What this screenshot proves:

- INPUT_BUCKET = sujal-input-images — Lambda reads images from here
- OUTPUT_BUCKET = sujal-output-images — Lambda saves processed images here
- SNS_TOPIC_ARN = arn:aws:sns:us-east-1:669493203560:image-processing-updates — Lambda notifies here
- No hardcoded values in code — professional configuration management

SCREENSHOT LOCATION



03 API Gateway — Invoke URL

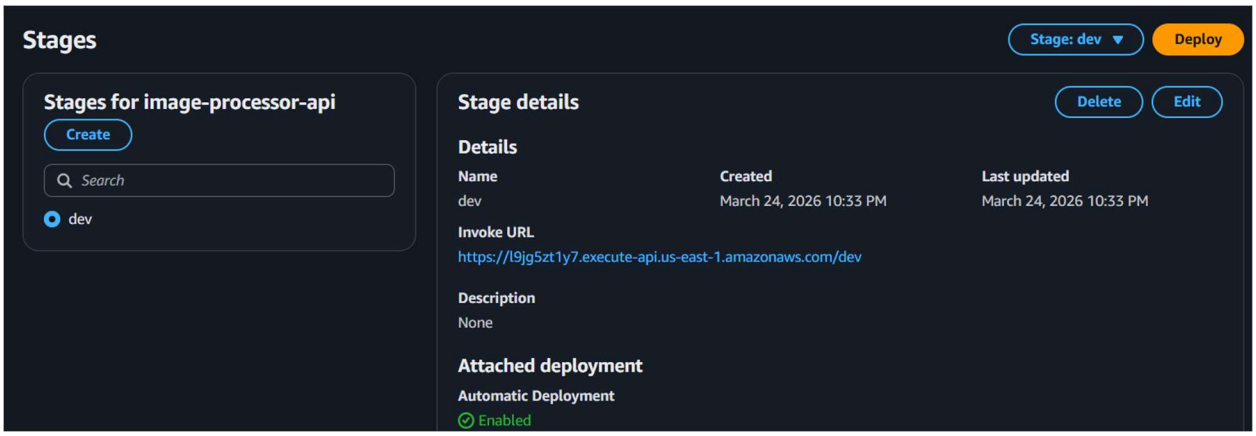
Where: API Gateway Console → image-processor-api → Stages → dev

The Invoke URL is the public address of your project. It is the URL that any application, browser, or tool like Postman uses to trigger your image processing pipeline over the internet. This screenshot proves that your API is deployed and has a real, accessible endpoint.

What this screenshot proves:

- The Invoke URL is visible at the top — this is the public HTTP address of your API
- Stage name 'dev' or '$default' confirms the API is deployed
- API name 'image-processor-api' is shown
- Auto-deploy is enabled — any changes deploy instantly

SCREENSHOT LOCATION



04 Postman — Successful API Test

Where: Postman Application

The Postman screenshot is the most powerful proof in this document. It shows that a real external HTTP request was sent to your API Gateway endpoint and received a successful response — proving the complete API Gateway → Lambda → S3 pipeline works end to end via a real-world HTTP call, exactly like a production application would use it.

What this screenshot proves:

- Method is POST — correct HTTP method for sending data
- URL shows your API Gateway Invoke URL — proves the public endpoint is working
- Body tab shows raw JSON: {"image_name": "sujal.jpeg"} — the filename sent to Lambda
- Response shows 200 OK — success status code from Lambda
- Response body shows JSON with message, processed_file, variants — Lambda's return value

Postman Setting	Value
Method	`POST`
URL	`https://l9jg5zt1y7.execute-api.us-east-1.amazonaws.com/process-images`
Body type	`raw → JSON`
Body content	`{"image_name": "sujal.jpeg"}`
Expected response	`200 OK with JSON body`

05 S3 Output — Three Folders

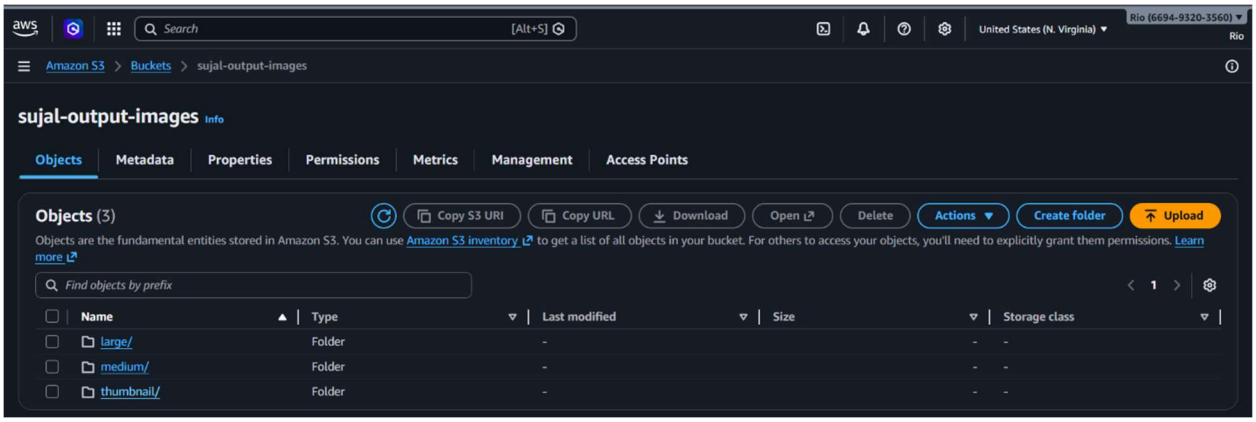
Where: S3 Console → sujal-output-images bucket

The S3 output structure screenshot proves that Lambda successfully processed the uploaded image and saved the results in an organized folder structure. Three folders — thumbnail, medium, large — are created automatically by the Lambda function. No manual folder creation was done by the user.

What this screenshot proves:

- thumbnail/ folder — contains image resized to max 150x150 pixels
- medium/ folder — contains image resized to max 500x500 pixels
- large/ folder — contains image resized to max 1024x1024 pixels
- All three created automatically by Lambda — proves automation is working
- Bucket name 'sujal-output-images' visible in the breadcrumb

SCREENSHOT LOCATION



06 Watermark Result

Where: Open processed image from thumbnail/ folder

The watermark screenshot is the visual proof that the Lambda function performed real image processing — not just copying files. The watermark 'Sujal | AWS Project' appears on a dark background box at the bottom-right corner of every processed image, clearly stamping ownership and proving the Pillow library is working correctly.

What this screenshot proves:

- Image is visibly smaller than the original — resize is working
- 'Sujal | AWS Project' text is visible at the bottom-right corner
- Dark background box behind the text makes it readable on any image
- Image quality is good — 75% JPEG compression maintained visual quality
- Proves Pillow library (Lambda Layer) is functioning correctly

SCREENSHOT LOCATION

07 SNS Email Notification

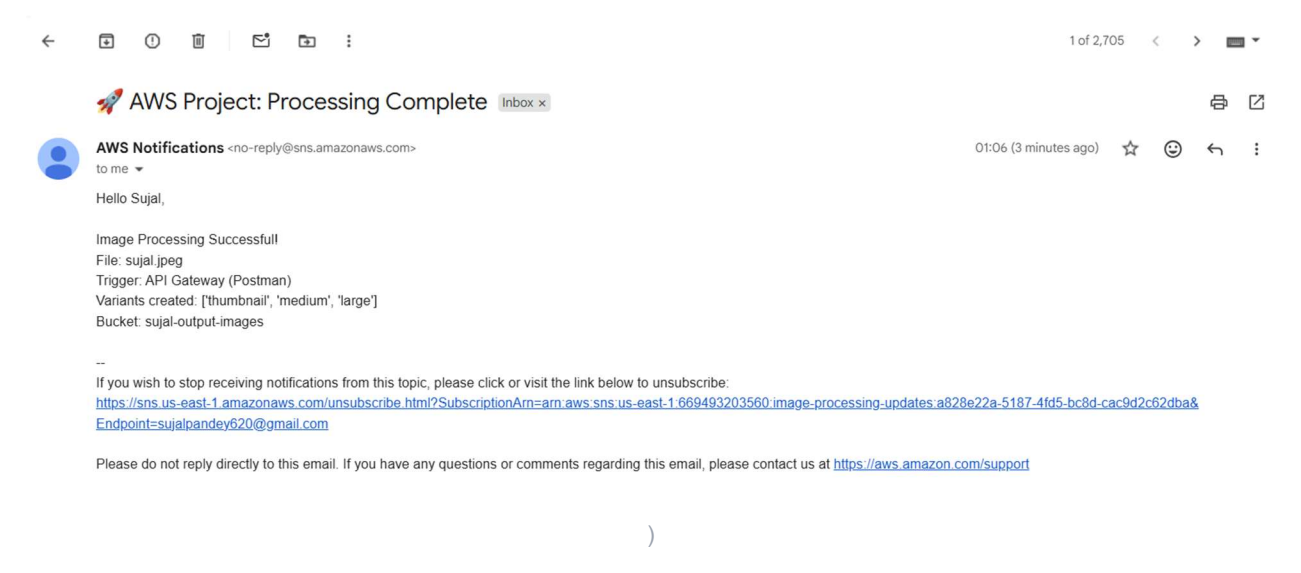
Where: Your Email Inbox

The SNS email notification is the final piece of evidence. It proves that after Lambda successfully processed the image, it published a message to the SNS topic, which then delivered an email to your inbox. This demonstrates the complete end-to-end pipeline including the notification layer — the hallmark of a production-grade serverless application.

What this screenshot proves:

- Email is from AWS SNS — sender shows no-reply@sns.amazonaws.com
- Subject confirms successful processing
- Email body shows the filename that was processed (sujal.jpeg)
- Output bucket name and folder structure mentioned
- Timestamp shows it arrived within seconds of the image upload — proving automation speed

SCREENSHOT LOCATION



Week 2 — Complete

7 screenshots | API Gateway + SNS + Postman + S3 + Lambda + Watermark

Total AWS Cost: $0.00 | Region: us-east-1 | Student: Sujal Pandey